IMPLEMENTATION PLAN

1D-CNN Solar Forecasting + Forecast Uncertainty + Dynamic Evolutionary Game Theory

Research scenario
A renewable-energy community of household prosumers chooses between two strategies: **Share** (participate in peer-to-peer energy trading) and **Hoard** (keep available energy private and use the grid/feed-in route when applicable). A 1D-CNN forecasts solar generation; forecast uncertainty and changing conditions update the strategy payoffs; replicator dynamics models the evolution of the sharing population.

Purpose of this document
A practical, staged plan to obtain data, prepare it, build and validate the forecasting model, estimate uncertainty, implement the dynamic evolutionary game, and evaluate whether the proposed interaction produces defensible research findings.

Important scope note
The commonly used Kaggle Solar Power Generation dataset contains a generation file and a separate weather-sensor file. Use the matching plant files together where timestamps permit. The generation file alone does not contain weather measurements.

Prepared: 9 October 2026

1. Research design at a glance

The paper should demonstrate more than simply joining a CNN and a game. Its central hypothesis is that **forecast uncertainty changes the relative payoff of Share versus Hoard, and those time-varying payoffs change evolutionary outcomes**. Treat this as a hypothesis to test, not a result to assume.

Component	Plan
Players	N household prosumers in one renewable-energy community.
Strategies	Share: participate in P2P exchange. Hoard: retain available energy for own use/storage and do not participate in P2P exchange.
Forecast target	Plant-level aggregated AC power, at 15-minute resolution; forecast 96 future intervals for a 24-hour horizon.
Forecast inputs	Past generation and, if matched correctly, weather-sensor features such as irradiation, ambient temperature and module temperature; add time-of-day cyclic features.
Uncertainty	Estimate predictive spread with Monte Carlo dropout or a small deep ensemble; validate interval coverage and width.
Game	Two-strategy evolutionary game with time-varying, uncertainty-aware Share and Hoard payoffs.
Main outcomes	Forecast accuracy; uncertainty calibration; Share fraction over time; equilibrium/stability; P2P energy traded; community cost/savings; sensitivity to uncertainty and weather shocks.

Recommended high-level pipeline

Generation CSV + Weather Sensor CSV → timestamp alignment and quality checks → plant-level time series → 1D-CNN 24-hour forecast → uncertainty estimation → dynamic Share/Hoard payoffs → replicator dynamics → forecast/game/economic evaluation.

Critical limitation of the dataset

The generation dataset covers roughly 34 days. This is useful for a prototype but short for strong claims about year-round generalisation, seasonal effects, or robust rare-weather events. State this limitation clearly. For stronger evidence, add a longer-span weather/solar source or a second independent dataset and evaluate it separately.

2. Data sources and resource links

Start with the Kaggle dataset already inspected. Its generation data is sampled every 15 minutes and includes inverter-level records. The dataset page also lists a separate weather-sensor file and Plant 2 files; inspect the files available in the download before deciding which to combine.

Primary dataset — Kaggle Solar Power Generation Data

Kaggle notebook/data view listing generation and weather-sensor files

Useful additional sources

PVGIS hourly radiation and PV output — official European Commission resource

PVGIS API documentation — programmatic data access

National Solar Radiation Database (NSRDB) — official download/API documentation

TimeSeriesSplit — time-ordered validation documentation

Keras Dropout layer — useful if using Monte Carlo dropout

TensorFlow/Keras Conv1D documentation

Which data files to download

- **Plant_1_Generation_Data.csv** — inverter-level DC_POWER, AC_POWER and yield records.
- **Plant_1_Weather_Sensor_Data.csv** — inspect its actual columns; the common version contains ambient temperature, module temperature and irradiation. Verify the timestamp format and sampling frequency rather than assuming they match perfectly.
- **Plant_2_Generation_Data.csv** and **Plant_2_Weather_Sensor_Data.csv** — optional second plant for a separate generalisation test; do not mix plant identities blindly.

Access notes

Kaggle may require a free account and acceptance of its dataset terms. Download the CSV files manually or use the Kaggle API after configuring credentials. Keep a copy of the dataset citation, version/date accessed, and any licence/usage notes in the project repository.

3. Dataset preparation

Step 1 — inspect and align timestamps

- Parse DATE_TIME into a real datetime type, sort ascending, and check duplicates.
- Check whether each timestamp is spaced at 15-minute intervals. Identify missing intervals and duplicate inverter-timestamp pairs.
- Merge generation and weather data on timestamp only after confirming both refer to the same plant and local time convention.
- Create a complete 15-minute time index. Do not silently interpolate long gaps; record how many values were missing and use a documented short-gap policy.

Step 2 — aggregate inverter generation

For plant-level forecasting, aggregate all inverter records at each timestamp. A suitable starting target is the sum of AC_POWER across SOURCE_KEY values. Check units and confirm that rows are inverter-level readings, not already plant totals.

```
AC_t = Σ■ AC_POWER■,t
```

Use AC_POWER as the main target because it represents inverter AC output. DC_POWER can be an input feature if it is available at the forecast origin; for a genuine day-ahead forecast, future DC_POWER is not available and must never be used as a future input.

Step 3 — features and leakage checks

Feature	Recommendation
AC_POWER history	Use lagged values only; it is also the target.
DC_POWER history	Optional lagged input if it adds value; test ablation.
Irradiation / temperature	Use weather observations only up to forecast origin. For a true 24-hour-ahead operational forecast, future weather inputs should be weather forecasts, not actual future sensor observations.
Hour/day cyclic features	Use sine/cosine encodings for time of day; optionally day-of-year if enough data exists.
DAILY_YIELD	Cumulative within-day value; consider excluding initially or converting to a safe lagged/derived feature after leakage analysis.
TOTAL_YIELD	Generally exclude: mostly cumulative, identifier-like and unlikely to provide useful forecast signal.
PLANT_ID	Drop if constant in the selected file.
SOURCE_KEY	Use to aggregate inverter data; drop after plant-level aggregation unless building a per-inverter model.

Step 4 — scaling

Fit feature scalers on the training period only, then apply the fitted transformation to validation and test periods. Do not fit scalers on the entire dataset, as this leaks information from the future.

4. Forecasting model specification

Initial experimental setup

Design choice	Starting value
Sampling	15 minutes; 96 intervals per day.
Input history	48 hours = 192 time steps; also test 24-hour (96-step) history as an ablation.
Forecast horizon	24 hours = 96 future values.
Input tensor	(samples, 192, number_of_features).
Output tensor	(samples, 96) for plant-level AC_POWER.
Model	A compact Conv1D stack with ReLU, pooling or strided convolution, dropout, flatten/global pooling, and a dense output head of 96 values.
Loss	MAE or Huber loss; choose using validation results.
Optimiser	Adam with early stopping on validation loss.
Baselines	Persistence/last-day profile, same-time-previous-day, and a simple non-neural baseline such as Random Forest or gradient boosting if feasible.

Temporal split

Use chronological splits, not random shuffling. Example starting split: earliest 70% for training, next 15% for validation, latest 15% for test. Build all windows after sorting; ensure a training target window does not overlap the test target period. With limited data, report the short test duration and use rolling-origin evaluation where feasible.

Forecast metrics

Metric	Purpose / caution
MAE	Mean absolute error; easy to interpret in the target's power units.
RMSE	Penalises large errors more heavily than MAE.
nMAE / nRMSE	Normalise by a clearly stated plant capacity or training-set scale to compare scales.
R^2	Supplementary only; can be misleading for strongly diurnal/zero-heavy output.
Daylight-only MAE/RMSE	Report alongside all-interval scores because night-time zeros can make overall error look artificially good.
Per-horizon error	Report error by lead time (e.g., 15 min, 1 h, 6 h, 12 h, 24 h).

Do not claim that a 1D-CNN is superior unless it beats credible baselines on the held-out chronological test period.

5. Forecast uncertainty

The ordinary deterministic CNN returns a point forecast only. It does not automatically provide a valid σ. For the proposed work, use a method that produces multiple plausible forecasts or prediction intervals and evaluate their calibration.

Recommended first method: Monte Carlo dropout

1. Keep dropout active during inference and run the same input through the trained model K times (start with K = 50–100).
2. For each future interval h, calculate the mean prediction across runs as the point forecast and the standard deviation across runs as an uncertainty proxy.
3. Because dropout spread alone may not capture all forecast error, calibrate intervals on the validation period using residual quantiles or a documented calibration method.
4. Clip physically impossible negative solar output to zero only after preserving raw predictions for diagnostics; explain this post-processing.

Alternative: deep ensemble

Train 3–5 independently initialised CNNs and use their prediction spread as an uncertainty proxy. Ensembles are often more robust but require more training. Compare both only if compute/time allows.

Uncertainty metrics

Metric	Definition / use
PICP	Prediction Interval Coverage Probability: fraction of actual targets inside the interval; compare with nominal 80% or 90% coverage.
MPIW	Mean Prediction Interval Width; lower is sharper, but only meaningful alongside coverage.
Pinball loss / CRPS	Optional probabilistic scores if the model is extended to quantiles or a full predictive distribution.
Coverage by lead time	Check whether intervals remain calibrated from near-term to 24-hour horizons.

Important distinction

Use σ in consistent units or normalise it (for example by rated plant capacity or a training-only target scale). Do not set arbitrary uncertainty percentages and present them as model estimates. Synthetic uncertainty sweeps can be used as sensitivity experiments, but keep them separate from the measured forecast uncertainty experiment.

6. Dynamic evolutionary game

Players and strategies

Players are household prosumers, i = 1,…,N. Each household chooses Share or Hoard. Let x(t) be the population fraction choosing Share; 1 – x(t) is the fraction choosing Hoard. The utility grid is an external market in this version, not a third strategy or player.

Baseline payoff (keep as a baseline, not automatically as the final economic model)

The earlier blueprint used a Share payoff depending on the current Share fraction and forecast generation, and a Hoard payoff based on the feed-in tariff. Before using these equations in a paper, check units and energy-market consistency: power in kW multiplied by a price in currency/kWh is not revenue unless converted to energy over a time interval.

```
For interval length Δt = 0.25 h, an energy estimate is E_t ≈ P_t × Δt (kWh). Revenue/cost terms
should be in a consistent currency per interval or per day.
```

Candidate dynamic payoffs

Define a time-varying payoff using forecast generation, forecast uncertainty, current participation and time-varying market conditions:

```
U_S(t) = Revenue_S(t) – Cost_S(t) – λ_S R_t
U_H(t) = Revenue_H(t) – Cost_H(t) – λ_H R_t
```

where R_t is a calibrated risk measure derived from forecast uncertainty, for example variance of the predictive distribution or expected shortfall. λ_S and λ_H must be justified from a behavioural/economic interpretation or treated as sensitivity parameters. Do not assume λ_S > λ_H without testing and explaining why the strategies bear different risks.

Replicator dynamics

```
dx/dt = x(t)[1 – x(t)] [U_S(t) – U_H(t)]
```

For implementation, discretise with a stable step size and clamp x to [0,1]. Since payoffs and forecasts change over time, this is a non-autonomous/time-varying evolutionary system. Report the numerical step size and verify that results are not artefacts of it.

Economic model requirements

- Define who has surplus and deficit energy, how much energy can actually be traded, and what happens when community supply does not meet demand.
- Specify P2P price, grid import tariff, feed-in tariff, transaction/platform cost, battery assumptions, and time interval.
- Avoid paying every household on aggregate plant generation without allocating generation and demand to players. If household-level data are unavailable, state that the game is a stylised population model and clearly separate plant-level forecast validation from household-level simulated economics.
- Make dynamic payoffs change for a defensible reason (forecast, uncertainty, demand, prices, participation), not just by inserting time indices into static equations.

7. Experiments and result tables

Run controlled comparisons so each proposed component can be credited for its effect.

ID	Model	Purpose
M0	Persistence / same-time-previous-day forecast	Establish a credible forecast baseline.
M1	Deterministic 1D-CNN + baseline game	Test the base forecasting/game pipeline.
M2	1D-CNN + dynamic payoff, no uncertainty penalty	Isolate the effect of time-varying payoff.
M3	1D-CNN + uncertainty-aware dynamic payoff	Test the main proposed extension.
M4 (optional)	M3 with alternative uncertainty method or payoff-risk function	Robustness / ablation test.

Primary evolutionary/game metrics

- **Share adoption x(t)** and final Share fraction x(T).
- **Equilibrium/stability:** whether trajectories converge, oscillate or change with weather/forecast conditions; for time-varying payoffs, do not assume a single fixed equilibrium exists.
- **Time above cooperation threshold:** e.g., fraction of simulated time with $x(t) \geq 0.8$; label the threshold as an analysis choice.
- **Critical uncertainty / tipping point:** estimate only if results show a repeatable transition; report sensitivity to λ and market assumptions.
- **Weather-shock response:** change in x(t), recovery time after the shock, and cumulative P2P trade lost during the shock.

Community/economic metrics

- P2P energy traded (kWh) and share of available surplus traded.
- Grid imports (kWh) and grid-dependency ratio, with denominator explicitly defined.
- Total community cost and savings versus a clearly defined no-P2P baseline.
- Renewable self-consumption/utilisation, curtailment or exported energy if the data/model supports those quantities.
- Report uncertainty intervals across multiple simulation runs or parameter settings; do not report only one favourable trajectory.

Suggested figures for the paper

1. Actual versus predicted plant AC power for representative test days.
2. Forecast error by lead time and daylight versus night-time.
3. Prediction interval plot and uncertainty calibration (coverage versus nominal level).
4. x(t) for M1, M2 and M3 under the same initial condition and prices.
5. Final Share fraction versus measured/simulated uncertainty level, with parameter sensitivity.
6. Share fraction and community outcomes before, during and after a simulated cloudy/rainy shock.

8. Practical implementation schedule

Phase	Tasks	Deliverable / exit check
1. Data audit	Download files; inspect columns, timestamps, gaps, duplicates, units and license.	Data dictionary + quality report.
2. Prepare series	Merge matching weather data; aggregate inverter AC_POWER; engineer time features; create chronological split.	Cleaned time series and reproducible preprocessing script.
3. Baselines	Implement persistence and same-time-previous-day forecast; calculate test metrics.	Baseline metric table.
4. CNN	Build Conv1D model; tune only on validation; save scaler/model/config.	Trained 96-step forecast model.
5. Uncertainty	Use MC dropout or ensemble; calibrate intervals; evaluate PICP/MPIW.	Forecast + uncertainty outputs.
6. Game model	Specify energy balance, price/cost units, baseline payoffs and dynamic risk-aware payoffs.	Documented equations and parameter table.
7. Simulation	Implement replicator dynamics, shock scenarios, uncertainty sweeps and ablations.	Reproducible simulation results.
8. Evaluation	Run all baselines; sensitivity checks; aggregate tables/plots; document limitations.	Final figures, tables and experiment log.
9. Paper	Write data/methodology, results, limitations, reproducibility and contribution claims.	Research manuscript with code/data citation.

Recommended software stack

Python; pandas and NumPy for data; matplotlib for plots; scikit-learn for scaling and chronological validation; TensorFlow/Keras or PyTorch for Conv1D; SciPy or a transparent custom solver for numerical dynamics; Jupyter notebooks for exploration; Git for version control.

Suggested repository structure

```
project/
data/raw/ (do not commit restricted/raw files if terms disallow)
data/processed/
notebooks/01_data_audit.ipynb
src/preprocess.py
src/train_cnn.py
src/uncertainty.py
src/payoffs.py
src/replicator.py
src/experiments.py
results/figures/ results/tables/
configs/ README.md requirements.txt
```

9. Risks, checks and research integrity

- **Short observation period:** roughly 34 days is not enough to substantiate year-round performance. Avoid generalising beyond the observed conditions.
- **Weather-data mismatch:** check that the weather sensor file corresponds to the same plant and timestamps. Never use actual future weather readings as if they were known day-ahead forecasts.
- **Power versus energy:** convert kW to kWh using the 0.25-hour interval before computing revenues or traded energy.
- **Uncertainty validity:** dropout variance is a proxy, not automatically a calibrated predictive standard deviation. Validate coverage and width.
- **Game assumptions:** payoff signs and strategy labels must match the actual economics; sharing is not guaranteed to dominate hoarding just because the P2P price is higher.
- **Identifiability:** plant-level solar data does not reveal individual household loads, battery states, or actual P2P trades. Those must be sourced, measured, or explicitly simulated.
- **Fair comparisons:** use the same held-out period, starting population, prices and scenario assumptions for M1/M2/M3.
- **Reproducibility:** fix random seeds where possible; save versions, splits, parameters and trained artefacts; report negative results as well as positive results.

What would count as convincing evidence?

The paper is stronger if (1) the CNN improves over simple baselines on a chronological test set, (2) the uncertainty intervals are reasonably calibrated, (3) M3 differs from M2 in a repeatable and interpretable way, and (4) the resulting change in Share adoption corresponds to meaningful community outcomes under a clearly specified energy-market model. If these do not occur, report that honestly and revise the hypothesis rather than forcing a tipping-point claim.

10. Source list

Kaggle: Solar Power Generation Data

Kaggle notebook listing generation and weather files

European Commission JRC: PVGIS hourly radiation

European Commission JRC: PVGIS API

NLR: NSRDB downloads/API

scikit-learn: TimeSeriesSplit

TensorFlow: Conv1D

Keras: Dropout

Final recommendation: implement the project in stages. First establish a clean, leak-free 24-hour forecasting benchmark; then add and validate uncertainty; only then connect the forecast to a carefully dimensioned Share/Hoard economic model and test whether dynamic, uncertainty-aware payoffs change evolutionary behaviour.